Combination Sum

[2, 4, 6, 8], target = 8

[2, 2, 2, 2] [2, 2, 4] [2, 6]
[4, 4]
[8]

fun(i, arr, target):
    Take arr[i] -> fun(i, arr, target - arr[i])
    Skip arr[i] -> fun(i + 1, arr, target)

```
4   void combinationSumUtil(int index, vector<int> &arr, int target, vector<int> &curr, vector<vector<int>> &result) {
5       if (target == 0) {
6           result.push_back(curr);
7           return;
8       }
9       if (index == arr.size()) {
10          return;
11      }
12
13      // Take arr[i] inside our current combination.
14      if (arr[index] <= target) {
15          curr.push_back(arr[index]);
16          combinationSumUtil(index, arr, target - arr[index], curr, result);
17          curr.pop_back();
18      }
19
20      // Skip arr[i].
21      combinationSumUtil(index + 1, arr, target, curr, result);
22  }
```

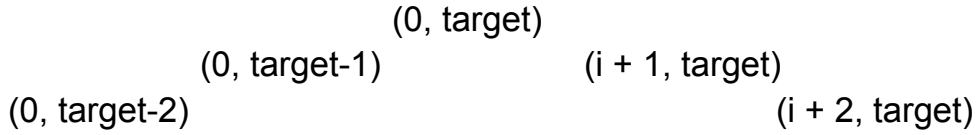
curr = {2, 2, 4}
result = {{2, 2, 2, 2}, {2, 2, 4}}
arr = {2, 4, 6, 8}

i = 0, target = 8

i = 0, target = 6

i = 0, target = 4

i = 1, target = 4



Length of the leftmost branch = target
Length of the rightmost branch = n

H = max(target, n)

TC: O(2^(max(target, n)))
AS: O(max(target, n)):
    Rec stack space: O(H) = O(max(target, n))
    Current combination = O(target)

D L R U

1 0 0 0
1 1 0 1
1 1 0 0
0 1 1 1

DDRDRR
DRDDRR

fun(i, j):
    Down: fun(i + 1, j) -> (+1, +0)
    Left: fun(i, j - 1) ->(+0, -1)
    Right: fun(i, j + 1) ->(+0, +1)
    Up: fun(i - 1, j) ->(-1, +0)

Move only if:
    The dest cell is valid and contains 1
    And, has not been visited in the current path.

1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1

H = n^2

TC: 4^(n^2)
AS: O(n^2)

Linked Lists

Just like array, linked list is also a linear DS.

```
Node {
        Int data;
        Node* next;
}
```

1, 2, 3, 4, 5

Array: [1, 2, 3, 4, 5]

Linked List:

Linked lists do not store elements in contiguous memory locations.

(head) **(tail)**
(1, 2x) -> (2, 3x) -> (3, 4x) -> (4, 5x) -> **(5, NULL)**

1 2 3 4 5

Time Complexities:
Insert at the beginning: O(1)
Insert at the end: O(n)
Delete from beginning: O(1)
Delete from the end: O(n)
Search: O(n)
Random Access to get the i-th node: O(i)

Arrays are cache-friendly but not Linked Lists -> Locality of references

arr[i] -> i, i + 1, i +2, i - 2, i -2 ..

L[i]

Insert at beginning:

data = 1

Input:
2->3->4->5->NULL

Output:
1->2->3->4->5->NULL

1->2->3->4->5->NULL

Create the node
newNode->next = head;
head = newNode;

Input:
Output: **1**->NULL

1->NULL

1->2->3->**NULL**

1 2 3

Insert At End:

Input: **0**->1->2->3->NULL, insert 4
Output: **0**->1->2->3->4->NULL

0->1->2->3->4
Traverse till the tail node
Create a new node with value = 4
tail->next = newNode

Insert At Pos:

Input: **0**->1->2->3->4->5, pos = 3, value = 6
Output: **0**->1->2->6->3->4->5

0->1->2->6->3->4->5

Stop at curr=2
temp = curr->next (3)
curr->next = newNode
newNode->next = temp

0->1->2->6->3->4->5
6

Stop at curr=2
newNode->next = curr->next
curr->next = newNode

1->2->3->NULL
Pos = 100
Data = 4

O/p: 1->2->3->4